
CSC 412 – Operating Systems

Lab Session 02, Fall 2024

Tuesday, September 16th and Thursday, September 18th, 2024

What this Lab Session Is About

This lab is all about `struct` in C, and pointers to `struct` and arrays of (pointers to) `struct`.

1 Task 1: Separate Output Strings

You should write a program that takes as single argument a C string (0-terminated array of `char`) and prints out three separate output C strings storing respectively the digits, letters, and other characters of the input string (possibly with characters being repeated)

For example, the following execution

```
./task1 "b+29Xa) / 72A3XxX-["
```

should produce the following output:

```
Digits:  29723
Letters:  AbXaAXxX
Other:   +) / -[
```

Note that I entered my input string between quotes. I had to do that so that characters such as the open parenthesis ‘(’ or the space character would not be interpreted by the shell as part of a command.

2 Task 2: Partial Sorting

This time, you want to output a single string that lists first all the digits, then the letters, and finally the other characters.

For example, the following execution

```
./task1 "Ab+29Xa) / 72A3XxX-["
```

should produce the following output:

```
Output:  29723AbXaAXxX+) / -[
```

The challenge here is that you are not allowed to simply compute and then concatenate the category strings of Task 1: You must write your characters directly into the output string. Note that, in order to do this you may¹ have to do more than one pass over the input string.

¹By which I mean: I don’t believe that there a way to do it in one pass, but I would welcome being mind-blown by a solution doing just that.

3 Task 3: Range and List Definition

In the first two tasks, the ranges of character types were “hard-coded” in the task definition (“integers,” “letters,” etc.). What I would like you to do in this task is to define a data type to represent a set of character ranges and individual characters. This data type should store the number of ranges, and “from” and “to” characters, as well as a name/label and an index for the category.

For example, a custom category type for “letters and digits” could get the arbitrary index 5 and contain the following data:

- label: `lettersAndDigits`
- index: 5
- number of ranges: 3
- range 0 (digits):
 - from: 0
 - to: 9
- range 1 (uppercase letters):
 - from: A
 - to: Z
- range 2 (lowercase letters):
 - from: a
 - to: z

Another category for “operators and braces” could store the following data:

- label: `operatorsAndBraces`
- index: 6
- number of ranges: 1
- range 0 (for ‘(’, ‘)’, ‘*’, and ‘+’):
 - from: (
 - to: +
- Flat list of characters (the rest):
 - number of characters: 6
 - list: `[, ], (, ), {, }, -`

Note that it doesn’t matter in which order the elements of the flat list are listed.